AI NEWS AGGREGATOR

AI-powered collection, categorization and analysis of news

LAB PROJECT 1

Team Name: neo

Reg. No.: 23BCB7172, 23BCB7059

Name: Rishabh Raj, Nikhil Bisht

Project repository: github.com/rishabhcod/ai-news-aggregator

1. Problem Statement

The Problem

AI news is distributed across many sources. Manually checking websites, identifying relevant stories and organizing them is time-consuming.

The Proposed Solution

A single AI-powered platform that aggregates news, processes and categorizes articles, and presents information in an organized interface for easier discovery and analysis.

Goal: reduce information overload while keeping AI-related updates easy to discover.

2. Objectives

- Collect news from multiple sources through automated/API-based ingestion.
- Apply AI-based processing to improve organization of incoming news.
- Categorize articles so users can navigate relevant topics quickly.
- Provide searchable and readable news content through a web interface.
- Store news data locally for analysis and reporting.
- Generate metrics and visualizations to evaluate system output.

3. Key Features

□ News Aggregation

Collects news from multiple sources into one platform.

□ AI Processing

Uses AI/ML techniques to process and organize incoming news.

□ Categorization

Groups articles into meaningful categories.

Q Search & Organization

Makes collected content easier to find and browse.

□ Analytics

Provides metrics and result visualizations.

□ Docker Support

Includes containerization support for deployment.

4. System Architecture / Workflow



High-level flow of the implemented platform

5. Technology Stack

Python

Core application and processing

AI / Machine Learning

News processing and intelligent organization

SQLite

Local persistence for collected news

HTML / CSS / JavaScript

Web interface

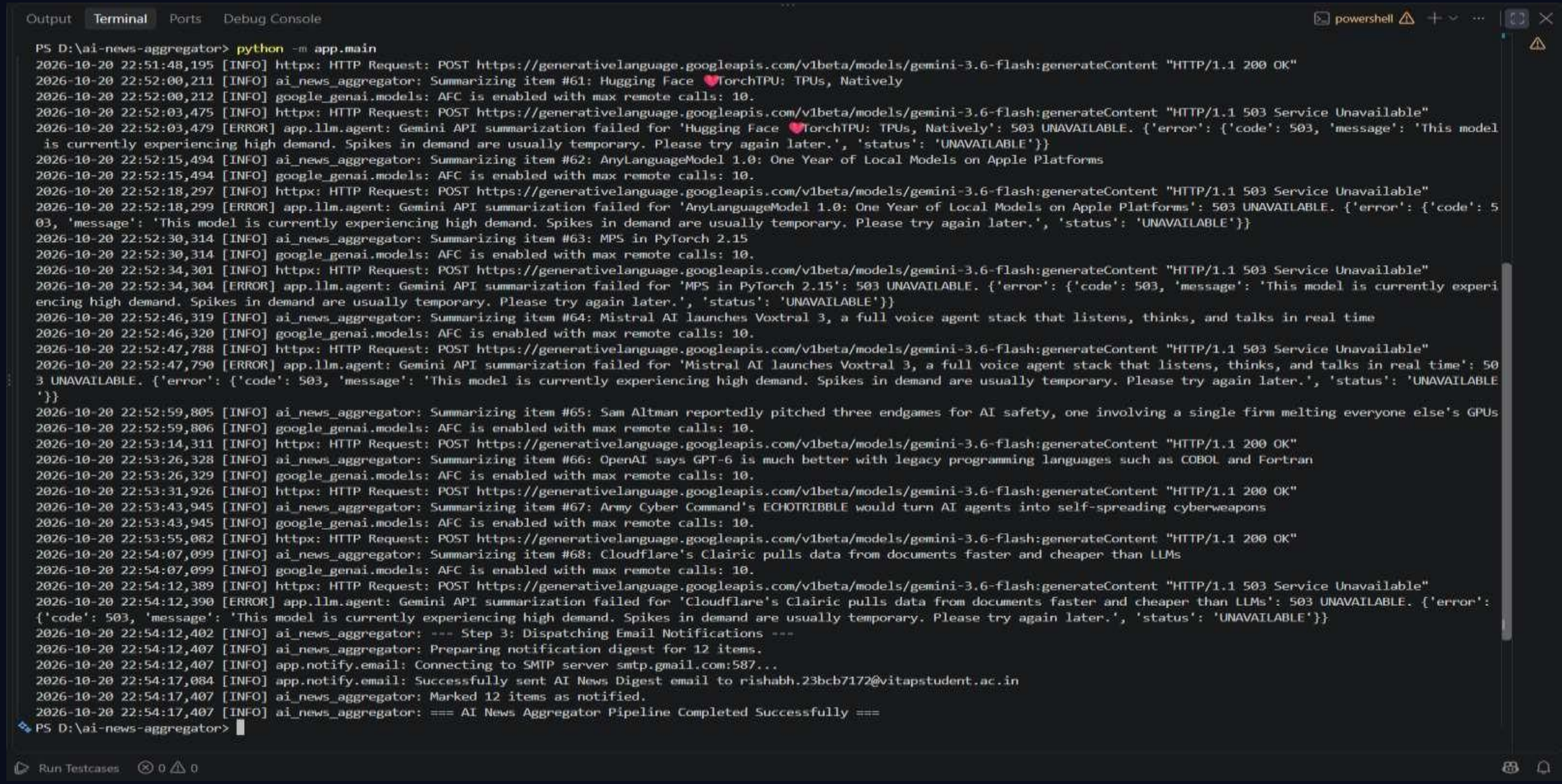
REST APIs

Data communication / integration

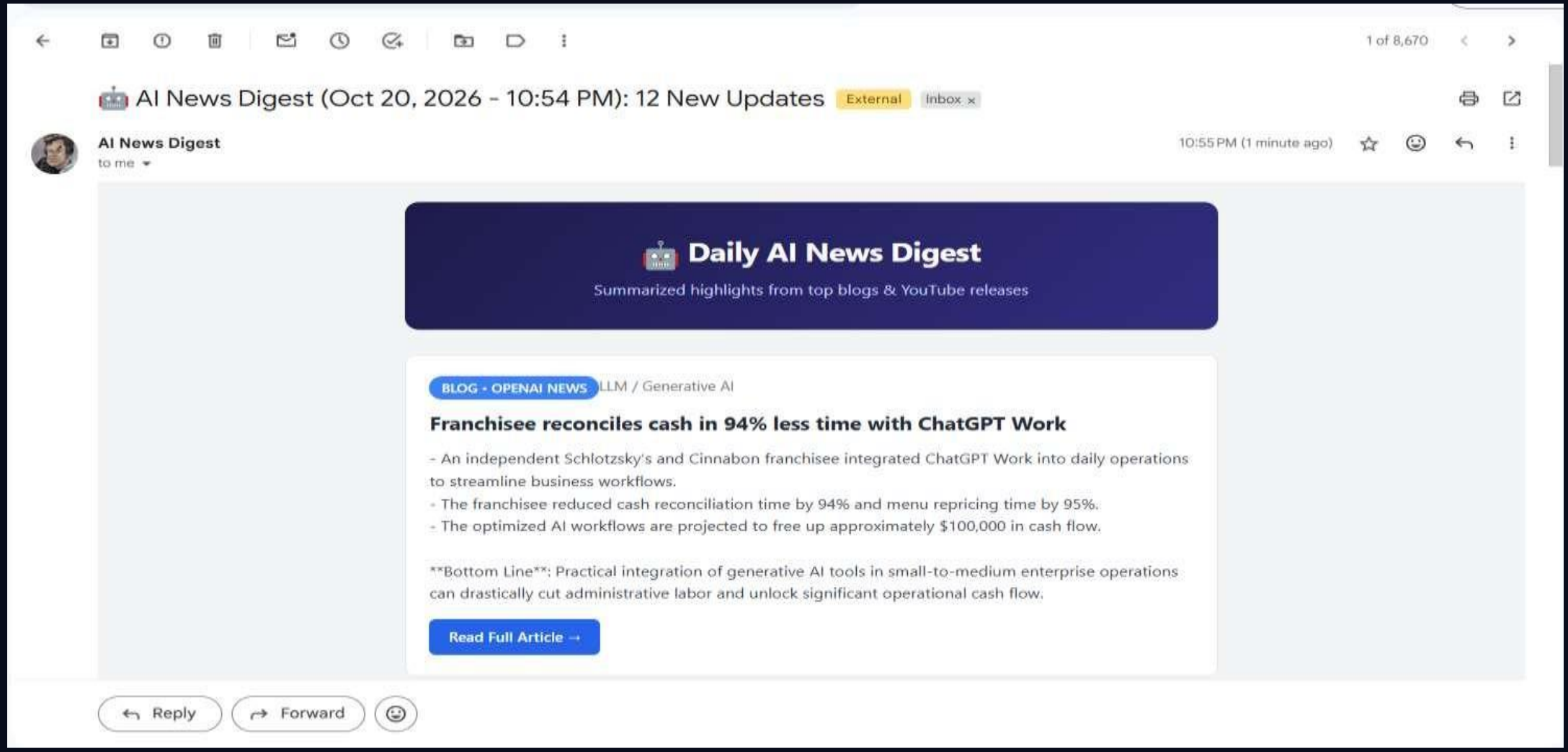
Docker

Containerization and deployment support

6. Project Implementation



7. Application Interface



8. Results & Benefits

Centralized Information

Brings news from multiple sources into a single platform, reducing the need to visit multiple websites.

Better Discoverability

Categorization, search and structured presentation make relevant articles easier to find.

Measurable Output

Metrics, charts and generated reports provide a way to evaluate and communicate system results.

Repository : https://github.com/rishabhcod/ai-news-aggregator

llm
__init__.py
agent.py
prompts.py
notify
__init__.py
email.py
__init__.py
config.py
database.py
main.py
models.py
docker
.env.example
.gitignore
README.md
ai_news.db
generate_pdf.py
generate_results_charts.py
project_presentation_guide.pdf


```python
import logging
import sys
from datetime import datetime
from sqlalchemy.orm import Session

from app.config import settings
from app.database import init_db, SessionLocal
from app.models import NewsItem, SourceType
from app.ingest import BlogIngester, YouTubeIngester
from app.llm import SummarizationAgent
from app.notify import EmailNotifier

# Setup logging
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
    handlers=[logging.StreamHandler(sys.stdout)],
)

logger = logging.getLogger("ai_news_aggregator")


def run_pipeline():
    """Main execution pipeline."""
    logger.info("=== Starting AI News Aggregator Pipeline ===")

    # 1. Initialize Database Schema
    logger.info("Initializing Database...")
    init_db()
    db: Session = SessionLocal()

    try:
        # 2. Ingest Latest News Items
```

Symbols

Find definitions and references for functions and other symbols in this file by clicking a symbol below or in the code.

Filter symbols R

const logger

func run_pipeline

llm
__init__.py
agent.py
prompts.py
notify
__init__.py
email.py
__init__.py
config.py
database.py
main.py
models.py
docker
.env.example
.gitignore
README.md
ai_news.db
generate_pdf.py
generate_results_charts.py
project_presentation_guide.pdf


```python
def run_pipeline():
    logger.info("--- Step 1: Ingesting Feed Data ---")

    # Blog feeds
    blog_ingester = BlogIngester(
        feed_urls=settings.blog_feed_list,
        max_items=settings.DEFAULT_MAX_ITEMS_PER_FEED
    )
    blog_items = blog_ingester.fetch_all()
    logger.info(f"Ingested {len(blog_items)} blog posts.")

    # YouTube feeds
    youtube_ingester = YouTubeIngester(
        channel_ids=settings.youtube_channel_list,
        max_items=settings.DEFAULT_MAX_ITEMS_PER_FEED
    )
    youtube_items = youtube_ingester.fetch_all()
    logger.info(f"Ingested {len(youtube_items)} YouTube videos.")

    all_fetched_items = blog_items + youtube_items
    new_inserted_count = 0

    # Save to database (deduplicate by URL)
    for item_data in all_fetched_items:
        existing = db.query(NewsItem).filter(NewsItem.url == item_data["url"]).first()
        if not existing:
            news_item = NewsItem(
                title=item_data["title"],
                url=item_data["url"],
                source_type=item_data["source_type"],
                source_name=item_data["source_name"],
                published_at=item_data["published_at"],
                raw_content=item_data["raw_content"],
                is_summarized=False,
```

Symbols

Find definitions and references for functions and other symbols in this file by clicking a symbol below or in the code.

Filter symbols R

const logger

func run_pipeline

llm
__init__.py
agent.py
prompts.py
notify
__init__.py
email.py
__init__.py
config.py
database.py
main.py
models.py
docker
.env.example
.gitignore
README.md
ai_news.db
generate_pdf.py
generate_results_charts.py

Code Blame 129 lines (108 loc) · 4.51 KB Raw

```
23  def run_pipeline():
67                      is_notified=False,
68                  )
69                  db.add(news_item)
70                  new_inserted_count += 1
71
72          db.commit()
73          logger.info(f"Successfully inserted {new_inserted_count} new news items into DB.")
74
75          # 3. LLM Summarization
76          logger.info("--- Step 2: Summarizing Pending Items ---")
77          pending_items = db.query(NewsItem).filter(NewsItem.is_summarized == False).all()
78          logger.info(f"Found {len(pending_items)} items pending LLM summarization.")
79
80          agent = SummarizationAgent()
81          import time
82          for idx, item in enumerate(pending_items):
83              logger.info(f"Summarizing item #{item.id}: {item.title}")
84              res = agent.summarize(
85                  title=item.title,
86                  source_name=item.source_name,
87                  source_type=item.source_type,
88                  raw_content=item.raw_content or "",
89              )
90              item.ai_summary = res["summary"]
91              item.category = res["category"]
92              item.is_summarized = True
93              db.commit()
94              if idx < len(pending_items) - 1:
95                  time.sleep(12)  # Respect Gemini Free-Tier 5 RPM rate limit
96
97          # 4. Dispatch Email Digest
98          logger.info("--- Step 3: Dispatching Email Notifications ---")
```

Symbols

Find definitions and references for functions and other symbols in this file by clicking a symbol below or in the code.

Filter symbols R

const logger

func run_pipeline

llm
__init__.py
agent.py
prompts.py
notify
__init__.py
email.py
__init__.py
config.py
database.py
main.py
models.py
docker
.env.example
.gitignore
README.md
ai_news.db
generate_pdf.py
generate_results_charts.py


```
23  def run_pipeline():
99          items_to_notify = db.query(NewsItem).filter(
100             NewsItem.is_summarized == True,
101             NewsItem.is_notified == False
102         ).all()
103
104         if items_to_notify:
105             logger.info(f"Preparing notification digest for {len(items_to_notify)} items.")
106             notifier = EmailNotifier()
107             sent_success = notifier.send_digest(items_to_notify)
108
109             if sent_success:
110                 for item in items_to_notify:
111                     item.is_notified = True
112                 db.commit()
113                 logger.info(f"Marked {len(items_to_notify)} items as notified.")
114             else:
115                 logger.warning("Email dispatch did not send (check credentials or SMTP config).")
116         else:
117             logger.info("No un-notified summarized items available.")
118
119         logger.info("=== AI News Aggregator Pipeline Completed Successfully ===")
120
121     except Exception as e:
122         logger.error(f"Pipeline error: {e}", exc_info=True)
123         db.rollback()
124     finally:
125         db.close()
126
127
128 if __name__ == "__main__":
129     run_pipeline()
```

Symbols

Find definitions and references for functions and other symbols in this file by clicking a symbol below or in the code.

Filter symbols R

const logger

func run_pipeline

9. Conclusion & Future Scope

Conclusion

The AI News Aggregator demonstrates how automation, AI-based processing, a local database and a web interface can be combined to turn scattered news into structured, accessible information.

Future Scope

- Add more news sources and richer source management
- Improve semantic relevance and duplicate detection
- Add personalized recommendations
- Introduce user accounts and saved preferences
- Expand analytics and deployment scalability

Thank You